

Devoir Java1 / AMRT2
Documents non autorisés / Durée : 3h

Exercice 1 : (3 points)

Écrivez un programme qui génère une matrice triangulaire inférieure avec n rangées et qui affecte aux éléments de cette matrice la somme de leurs positions sur la rangée et sur la colonne, en commençant par l'indice 0.

Voici un exemple d'exécution pour $n = 7$.

```
0
1 2
2 3 4
3 4 5 6
4 5 6 7 8
5 6 7 8 9 10
6 7 8 9 10 11 12
```

Exercice 2: (7 points)

- a) Ecrire une classe **Personne** permettant de décrire complètement une personne, sachant que l'on souhaite avoir autant d'informations que dans la phrase suivante :

"M. Malick Mbengue est né en 1990, il est célibataire."

- b) Ajouter un constructeur à la classe **Personne**.
- c) Ajouter, à la classe **Personne**, une méthode de nom **retourneInfos**. Cette méthode doit retourner une chaîne de caractères similaire à la phrase donnée dans a).
- d) Ecrire un programme qui déclare 3 variables de type **Personne**, crée 3 instances de **Personne** pour les affecter dans les variables et affiche les informations les concernant.
- e) Ajouter une méthode **age** qui renvoie l'âge de l'individu en fonction d'une année donnée en paramètre.
- f) Ajouter à la classe **Personne** un attribut **conjoint** et examiner les conséquences que cela peut avoir sur l'ensemble du code. Attention: la polygamie et la polyandrie sont interdites
- g) Ajouter une méthode **marier(Personne p)** qui permet de marier une personne à une autre. Modifier la méthode **retourneInfos** de façon que le nom (traditionnel de la femme devienne):

Pour simplifier : quand une femme se marie son nom devient : "[nom de l'époux] née [nom de jeune fille]", par exemple: si Mlle Faye se marie avec M. Mbengue, son nom deviendra Mme "Mbengue née Faye"

- * h) Ecrire un programme qui déclare 3 variables de type **Personne**, crée 3 instances de **Personne** pour les affecter dans les variables et affiche les informations les concernant.
- i) Ecrire une classe **Etudiant** qui hérite de **Personne**. Dans cette classe on ajoutera les informations suivantes: numéro d'étudiant, année de première entrée à l'université. Modifier la méthode **retourneInfos** pour prendre en compte ces nouvelles informations lors de l'affichage.

Exercice 3 : classes et héritage (6 points)

Nous disposons de la classe Point suivante :

```
class Point
{
    private double x,y;
    public Point (double x, double y){this.x=x; this.y=y;}
    public void deplace (double dx, double dy){ x+=dx; y+=dy;}
    public void affiche(){
        System.out.println("Point de coordonnées "+x+" "+y);
    }
    public double getX(){ return x; }
    public double getY(){ return y; }
}
```

On souhaite réaliser une classe Cercle, dérivant de point, définit par un centre et un rayon et disposant des méthodes suivantes :

- un constructeur recevant en argument les coordonnées du centre du cercle et son rayon,
- deplaceCentre pour modifier les coordonnées du centre du cercle
- changeRayon pour modifier le rayon du cercle
- getCentre qui fournit un objet de type Point correspondant au centre du cercle
- affiche qui affiche les coordonnées du centre du cercle et son rayon

Ecrire un code java codant cette classe.

Exercice 4 : (4 points)

Ecrire une application Java qui montre chaque argument qu'elle reçoit sur la ligne de commande.

Chacun des arguments doit être affiché sur une nouvelle ligne, ainsi que sa longueur en caractères.

A la fin, l'application affichera la somme de ces longueurs.

FIN

Exercice 1 :

Donner l'affichage du programme suivant :

```
public class Exercice1 {
    public static void main(String[] args) {
        StringBuffer sb = null;
        String str[] = {"2365", "Rue Cote-St Luc", "H3X 2F9", "(514) 9113989"};
        StringBuffer res = new StringBuffer();
        a) sb = new StringBuffer(str[0]);
        b) sb.reverse();
        c) res.append(sb.toString()+"\n");
        d) sb = new StringBuffer(str[1]);
        e) int pos = sb.indexOf("St");
        f) sb.replace(pos, pos+2, "Saint");
        g) res.append(sb.toString()+"\n");
        h) sb = new StringBuffer(str[2]);
        i) sb.deleteCharAt(sb.indexOf(" "));
        j) res.append(sb.toString()+"\n");
        k) sb = new StringBuffer(str[3]);
        l) sb.insert(sb.length()-4, '-');
        m) res.append(sb.toString()+"\n");
        n) System.out.println(res);
    }
}
```

Exercice 2 :

Écrire un programme qui lit une suite de mots au clavier et qui les affiche triés par ordre alphabétique. On supposera que ces mots ne contiennent que des lettres non accentuées (majuscules ou minuscules). Le nombre de mots sera fourni en données et l'exécution se présentera ainsi :

Combien de mots ? 5

donnez vos mots

JavaScript

Pascal

BaSiC

Java

ADA

Liste par ordre alphabétique :

ADA

BaSiC

Java

JavaScript

Pascal

Notez bien que les mots sont affichés avec leur "casse" d'origine mais que celle-ci n'influe pas sur le tri qui respecte l'ordre alphabétique traditionnel (qui ne distingue pas les majuscules des minuscules).

Exercice 3 :

Écrire une application en Java qui enlève les espaces de la chaîne de caractères " Salut, voulez vous du café Touba? ". Et affiche à l'écran le résultat soit :

Salut,voulezvousducaféTouba?. Le programme devra fonctionner avec une chaîne quelconque.

Remarque : utiliser les deux classes StringBuffer et StringTokenizer

Exercice 4 : Classe abstraite

La classe **Etre_vivant** est une classe mère générale pour les êtres vivants sur la planète, chaque catégorie d'être vivant peut être représentée par une classe dérivée.

En partant du principe que les êtres vivants ci-dessous boivent tous de l'eau mais ne se déplacent pas de la même façon, définissez :

- Une Classe abstraite **Etre_vivant** avec les méthodes boire et déplacer (à vous de préciser la ou les méthodes abstraites)
- Les classes dérivées : **Serpent**, **Oiseau**, **Homme** en considérant qu'ils se déplacent respectivement (rampant, volant, marchant).
- Une classe de Test avec la méthode main

Exercice 5 : Exceptions

1. La déclaration d'une méthode suivante :

```
public void traitement(String s) throws MyException
```

L'affirmation suivante est-elle correcte : *oui*

Ce code précise que la méthode doit traiter l'exception MyException dans le corps de sa méthode.

2. Soit le code suivant qui ajoute un Individu dans un tableau :

```
public void ajouter(String nom) throws Exception  
{ Individu ind=null;  
  try{ ind = rechercher(nom); }  
  catch(NonTrouveException ex) { tab[n++] = ind; } }
```

La méthode rechercher retourne l'exception NonTrouveException si le nom de l'individu n'est pas trouvé.

Trouver la ou les bonnes réponses :

- a) si l'individu que l'on veut ajouter n'est pas trouvé alors la méthode retourne l'exception NonTrouveException
- * b) si l'individu que l'on veut ajouter n'est pas trouvé et le tableau n'est pas plein alors l'individu est ajouté au tableau
- c) Si l'individu que l'on veut ajouter n'est pas trouvé et le tableau est plein alors la méthode retourne une exception prédéfinie Java (IndexOutOfBoundsException)

3. - Soit le code suivant :

```
try{System.out.println("AAA");  
  call();  
  System.out.println("BBB"); }  
catch(MyException ex) { System.out.println("DDD"); }  
catch(Exception ex) { System.out.println("CCC"); }  
avec la méthode call qui déclenche l'exception MyException.
```

Ce code affiche :

- a) AAA BBB
- * b) AAA DDD CCC
- c) AAA DDD

Exercice 1 : Méthode main

Écrivez un programme qui convertit une chaîne de caractères représentant un nombre décimal en un nombre qui sera présenté sous le format de la monnaie courante du système. Affichez le résultat obtenu. La chaîne de caractères doit être fournie dans la ligne de commande.

Voici un exemple d'exécution :

C:\>java ConversionStrMonnaie 154.48

Montant en argent : 154,48\$

Exercice 2 : Palindrome

Un palindrome est un mot qui peut se lire de droite à gauche ou de gauche à droite. Par exemple le mot "elle" est un palindrome.

Écrire une classe Palindrome qui contient :

- ✕ • une méthode **public static boolean cmpCharAt(String s, int i, int j)**; qui compare le i-ème et le j-ème caractère de s et retourne true s'ils sont égaux.
- ✕ • une méthode **public static boolean palin(String s)**; contenant une boucle permettant de déterminer si un mot passé en paramètre est un palindrome.
- une méthode **public static boolean palinRec(String s, int i, int j)**; permettant de déterminer de façon récursive, si un mot passé en paramètre est un palindrome.
- une méthode **public static void main(String[] args)** ; de test de vos méthodes.

Exercice 3 : Racine carrée

1. Écrire une classe **Racine** servant à calculer la racine carrée d'un nombre réel de type **double**.

La classe **Racine** proposée contient un attribut privé (déclaré **private**), l'attribut **precision**, ce qui entraîne que cet attribut n'est pas directement accessible depuis une autre classe.

Il faut que la méthode **calculerRacine** qui permet de calculer la racine lance une exception dans le cas où le nombre **double** qui lui est passé en paramètre est négatif. Pour cela, on définira une classe étendant la classe **Exception**, correspondant à la classe de l'exception qui sera levée dans le cas d'un nombre négatif. Dans ce cas, on lancera une exception de la classe définie depuis l'intérieur de la méthode **calculerRacine** de la classe **Racine**. Cette erreur ne sera pas attrapée par la méthode **calculerRacine** mais le sera dans la méthode **main** qui utilise la méthode **calculerRacine**.

Pour le calcul de la racine carrée du nombre, on utilisera une méthode dichotomique. On initialisera l'intervalle sur lequel sera recherchée la solution avec l'intervalle [0, nombre] et la première solution dont le carré sera comparé au nombre sera le milieu de cet intervalle. On réitérera jusqu'à ce que la différence entre les deux bornes de l'intervalle soit inférieure ou égale à la valeur de la précision.

2. Écrire une classe **CalculRacine** qui permet d'utiliser la classe **Racine**. Le nombre à traiter sera envoyé comme argument sur la ligne de commande.

Exercice 4 : Classe abstraite

La classe **Etre_vivant** est une classe mère générale pour les êtres vivants sur la planète, chaque catégorie d'être vivant peut être représentée par une classe dérivée.

En partant du principe que les êtres vivants ci-dessous boivent tous de l'eau mais ne se déplacent pas de la même façon, définissez :

- Une Classe abstraite **Etre_vivant** avec les méthodes boire et déplacer (à vous de préciser la ou les méthodes abstraites)
- Les classes dérivées : **Serpent**, **Oiseau**, **Homme** en considérant qu'ils se déplacent respectivement (rampant, volant, marchant).
- Une classe de Test avec la méthode main

FIN

Licence 2 en AMRT
Examen Java / session 1
Durée : 2 heures

Questions de cours : (3pts)

Expliquez à quoi servent les exceptions et expliquer leur fonctionnement.

En programmation objet, donner une définition du polymorphisme.

Donner la différence entre la surdéfinition et la redéfinition de méthodes

Exercice 1 : (7 points)

On va définir une classe pour modéliser des marchandises. Chaque marchandise est définie par un nom et une valeur hors taxe en entier de CFA.

1. Définissez pour cette classe, deux attributs, sans valeur initiale : *value* un float et *name* une instance de la classe String.
2. Définissez deux constructeurs :
 - l'un n'a qu'un paramètre, il permet l'initialisation de l'attribut *name* et positionne l'attribut *value* à 0 par défaut. Quel est le type du paramètre ?
 - l'autre prend comme paramètre les valeurs d'initialisation des deux attributs *name* et *value*. Quels sont donc les types des paramètres ?
3. Ecrivez les méthodes d'accès (consultation) et de modification pour l'attribut *value*.
4. Idem pour uniquement l'accessor de *name*.
5. Codez une méthode *toString*, sans paramètre et qui retourne (pas affiche !) une chaîne de caractères dont la valeur est : "la marchandise *name* vaut *value*" (INFO : l'opérateur + permet la concaténation des chaînes de caractères avec conversion automatique des valeurs de types primitifs en valeurs de chaînes).
6. Codez une méthode qui retourne (pas affiche !) la valeur TTC de la marchandise, on prendra une TVA à 19.6% 1.
7. Testez.

Exercice 2: (4 points)

Écrivez un programme qui lit une série de nombres de type entier et qui affiche la somme de ces nombres. Les données doivent être lues à partir de la ligne de commande.

Voici un exemple d'exécution du programme :

C:\>java Somme 5 10 15 20 25 30 35 40 45 50

5 + 10 + 15 + 20 + 25 + 30 + 35 + 40 + 45 + 50 = 275

Exercice 3 : Héritage, classes abstraites et interfaces (6pts)

Les affirmations suivantes sont-elles correctes

1. L'héritage multiple de classe est possible en Java.
2. Une classe abstraite est une classe dont les types des attributs ne sont pas connus. C'est lors de l'instanciation de l'objet que les types des attributs sont résolus.
3. Soit deux classes B et C qui héritent d'une classe abstraite A.
Les classes B et C peuvent utiliser par héritage les attributs *protected* de la classe A.

4. Une classe qui contient au moins une méthode abstraite doit être déclarée abstraite.
5. Le programme suivant est-il correcte (1pt)

```
public class A extends B implements C
{ private int attr_A;

  public A() {
    attr_A = 10; attr_B = "TOTO"; C.attr_C = 100; }}
attr_B est un attribut protected de B
attr_C est un attribut static public de l'interface C
```

6. En JAVA, une interface peut contenir des attributs (cocher la ou les bonnes réponses).
a) statics
b) non statics
c) privés
7. La classe A hérite de B qui hérite de C. C est une classe abstraite qui implémente une interface I. A et B ne sont pas des classes abstraites et dans la classe c il n'ya pas de méthode abstraite. (Trouver la ou les bonnes réponses)
a) C peut implémenter qu'une partie des méthodes de l'interface I
b) B doit implémenter toutes les méthodes de I qui n'ont pas été implémentées par C
c) A ne peut pas implémenter des méthodes de l'interface I
8. On considère le code suivant:

```
class A {voidg() {System.out.println("A:g()"); }}
class B extends A {voidg() {System.out.println("B:g()"); }}
class C extends B {voidg() {System.out.println("C:g()"); }}
```

et la séquence de code **C a = new A();a.g();** quelle est la bonne réponse:

- a) Le code provoque une erreur à la compilation
b) Le code provoque une erreur à l'exécution
c) Il afficheC:g()
d) Il afficheB:g()

FIN